Name: Godwin M. Labaya **Date:** 05/09/2026
Course & Section: BSIT – 3 (IT-344 G5)

Full Regression Test Report

Project Information

DisasterAidConnect is a web-based disaster relief coordination platform designed to bridge the gap between affected communities, volunteers, and aid organizations during disaster situations. The system allows users to report disaster events by placing pins on an interactive map, submit aid requests for resources such as food, water, medical assistance, and shelter, and make financial donations to support relief efforts.

The platform features real-time disaster mapping using OpenStreetMap and Leaflet, user authentication via Supabase and Google OAuth2, and a structured aid request and donation management system. It aims to streamline disaster response by providing a unified platform where communities can report emergencies and receive coordinated assistance.

Refactoring Summary

The backend was refactored from a traditional layered architecture to Vertical Slice Architecture. Previously, the project was organized by technical layers (controller/, dto/, entity/, repository/, service/). Each layer contained files from all features mixed together, making it harder to navigate and maintain.

After refactoring, each feature is now self-contained in its own package, holding all the files it needs, entity, DTO, repository, service, and controller in one place.

Updated Project Structure

Updated Project Structure

Before (Layered Architecture):

```
edu.cit.labaya.disasteraidconnect
├── config/
|   └── SecurityConfig.java
├── controller/
|   └── DisasterController.java
├── dto/
```

```
|   ├── DisasterRequestDTO.java
|   └── DisasterResponseDTO.java
├── entity/
|   ├── AidRequest.java
|   ├── AidRequestImage.java
|   ├── Disaster.java
|   ├── Donation.java
|   ├── Payment.java
|   └── User.java
├── repository/
|   └── DisasterRepository.java
├── service/
|   ├── DisasterService.java
|   └── DisasterServiceImpl.java
└── strategy/
    ├── SeverityStrategy.java
    ├── LowSeverityStrategy.java
    ├── MediumSeverityStrategy.java
    ├── HighSeverityStrategy.java
    └── CriticalSeverityStrategy.java
```

After (Vertical Slice Architecture):

```
edu.cit.labaya.disasteraidconnect
├── disaster/
|   ├── Disaster.java
|   ├── DisasterController.java
|   ├── DisasterRepository.java
|   ├── DisasterRequestDTO.java
|   ├── DisasterResponseDTO.java
|   ├── DisasterService.java
|   └── DisasterServiceImpl.java
├── aidrequest/
|   ├── AidRequest.java
|   ├── AidRequestImage.java
|   ├── AidRequestController.java
|   ├── AidRequestRepository.java
```

```
|   ├── AidRequestRequestDTO.java
|   ├── AidRequestResponseDTO.java
|   ├── AidRequestService.java
|   └── AidRequestServiceImpl.java
├── donation/
|   ├── Donation.java
|   ├── DonationController.java
|   ├── DonationRepository.java
|   ├── DonationRequestDTO.java
|   ├── DonationResponseDTO.java
|   ├── DonationService.java
|   └── DonationServiceImpl.java
├── payment/
|   ├── Payment.java
|   └── PaymentRepository.java
├── user/
|   ├── User.java
|   ├── UserController.java
|   ├── UserRepository.java
|   └── UserResponseDTO.java
└── core/
    ├── config/
    |   └── SecurityConfig.java
    └── strategy/
        ├── SeverityStrategy.java
        ├── LowSeverityStrategy.java
        ├── MediumSeverityStrategy.java
        ├── HighSeverityStrategy.java
        └── CriticalSeverityStrategy.java
```

Test Plan Documentation

Scope

This test plan covers all implemented functional requirements of the DisasterAidConnect web platform after Vertical Slice Architecture refactoring.

Functional Requirements Coverage

FR-01 User Registration

FR-02 User Login (Email/Password)

FR-04 View Interactive Disaster Map

FR-05 Add Disaster Point on Map

FR-06 Edit Disaster Point (owner only)

FR-07 Delete Disaster Point (owner only)

FR-08 View Disaster Point Details

FR-09 Upload Proof Images for Disaster Point

FR-10 View My Requests Page

FR-11 Filter/Search Requests

FR-12 View Donations Page

FR-13 View About Page

FR-14 View Help & Support Page

FR-15 Logout

Test Cases

TC-01 | User Registration

Precondition: User is not registered

Steps:

1. Navigate to /register
2. Fill in username, email, password, confirm password, security question, and answer
3. Click SIGN UP

Expected Result: Account created, redirected to login page

Status: **PASS**

TC-02 | User Login - Valid Credentials

Precondition: User has a registered account

Steps:

1. Navigate to /
2. Enter valid email and password
3. Click SIGN IN

Expected Result: Redirected to /dashboard

Status: **PASS**

TC-03 | User Login - Invalid Credentials

Precondition: None

Steps:

1. Navigate to /
2. Enter wrong email or password
3. Click SIGN IN

Expected Result: Error alert shown, stays on login page

Status: **PASS**

TC-05 | View Interactive Disaster Map

Precondition: User is logged in

Steps:

1. Navigate to /map

Expected Result: Map loads with OpenStreetMap tiles and existing disaster markers

Status: **PASS**

TC-06 | Add Disaster Point

Precondition: User is logged in and on /map

Steps:

1. Click "Add Point" button
2. Click a location on the map
3. Fill in title, description, severity, status
4. Optionally upload up to 3 images
5. Click Save Point

Expected Result: New marker appears on map, point added to list

Status: PASS

TC-07 | Edit Disaster Point (Owner)

Precondition: User is logged in and owns the point

Steps:

1. Select own disaster point on map
2. Click Edit button
3. Modify title or description
4. Click Save Point

Expected Result: Point updated on map and in detail card

Status: **PASS**

TC-08 | Delete Disaster Point (Owner)

Precondition: User is logged in and owns the point

Steps:

1. Select own disaster point on map

2. Click Delete button

3. Confirm deletion in dialog

Expected Result: Point removed from map and list

Status: **PASS**

TC-09 | Non-Owner Cannot Edit or Delete

Precondition: User is logged in, point belongs to another user

Steps:

1. Select a disaster point not created by current user

2. Check detail card

Expected Result: Edit and Delete buttons are not visible

Status: **PASS**

TC-10 | Upload Proof Images

Precondition: User is adding a new disaster point

Steps:

1. In Add Point form, click "Add photo"

2. Select 1 to 3 image files

3. Save the point

Expected Result: Images uploaded to Supabase Storage,
thumbnails visible in detail card

Status: **PASS**

TC-11 | View My Requests

Precondition: User is logged in

Steps:

1. Navigate to /requests

Expected Result: Only disaster points created by the
current user are shown

Status: **PASS**

TC-12 | Search and Filter Requests

Precondition: User is on /requests with existing points

Steps:

1. Type keyword in search box
2. Select a status tab (Active / Monitoring / Resolved)
3. Pick a date filter

Expected Result: List filters correctly by all criteria

Status: **PASS**

TC-13 | Delete Request from Requests Page

Precondition: User is on /requests

Steps:

1. Click Delete on a request card
2. Confirm in dialog

Expected Result: Request removed from list

Status: **PASS**

TC-14 | Logout

Precondition: User is logged in

Steps:

1. Click Logout in sidebar

Expected Result: Session cleared, redirected to /login

Status: **PASS**

TC-15 | Protected Route Redirect

Precondition: User is not logged in

Steps:

1. Navigate directly to /dashboard

Expected Result: Redirected to / (login page)

Status: **PASS**

Automated Test Evidence

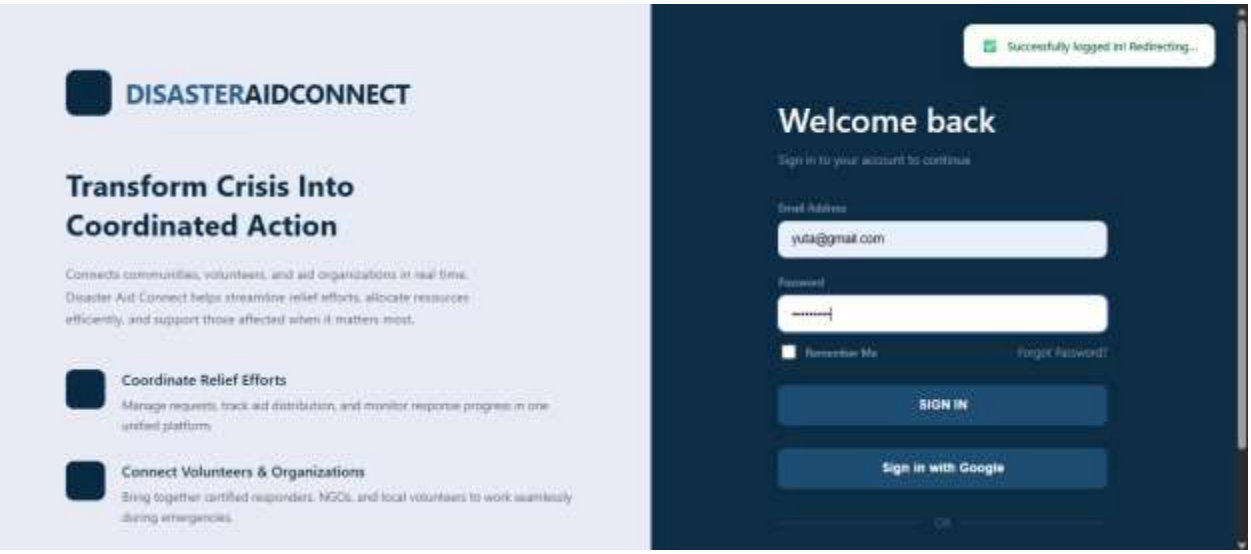
Testing Approach

Due to the scope and timeline of this capstone project, manual regression testing was the primary verification method used. Evidence was collected through browser DevTools, Supabase dashboard logs, and direct UI interaction screenshots for each test case.

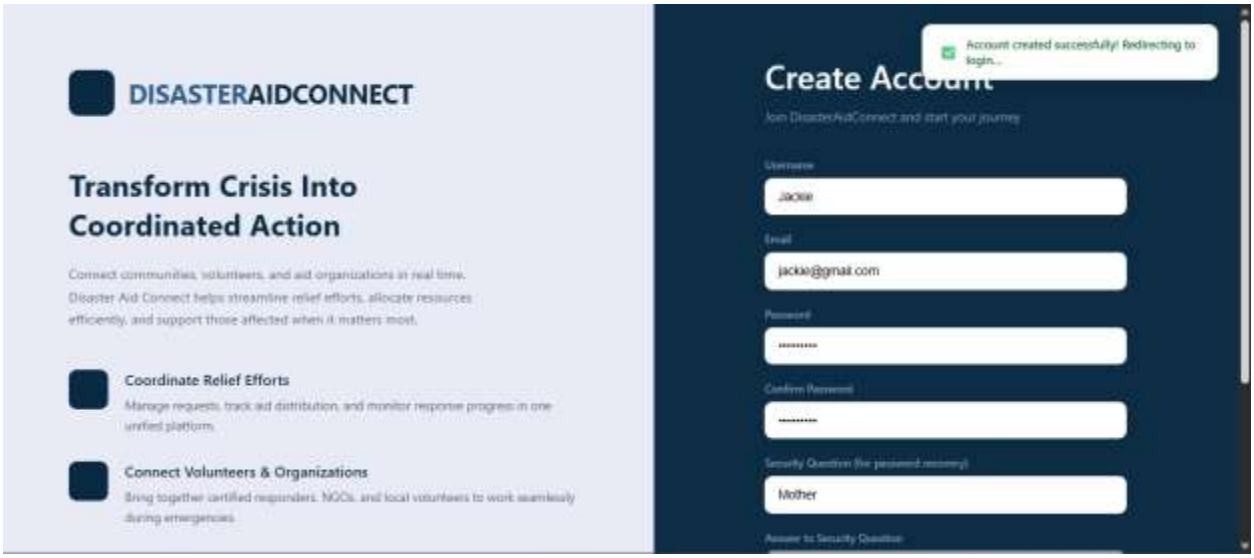
Test Execution Screenshots

Screenshots were captured for the following test scenarios:

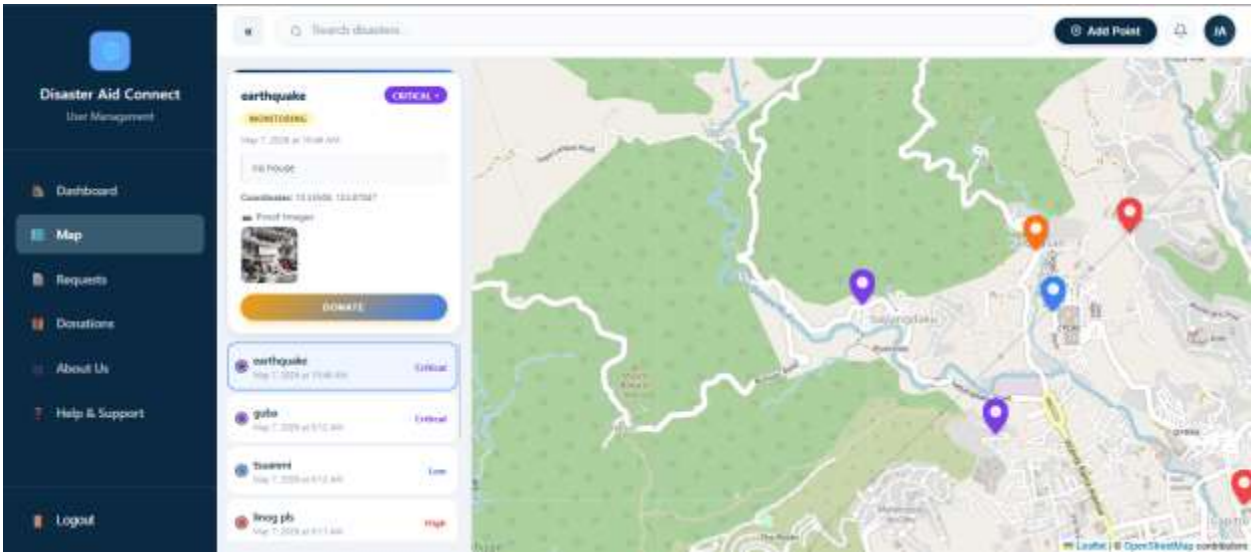
1. Login page — successful login redirect to /dashboard



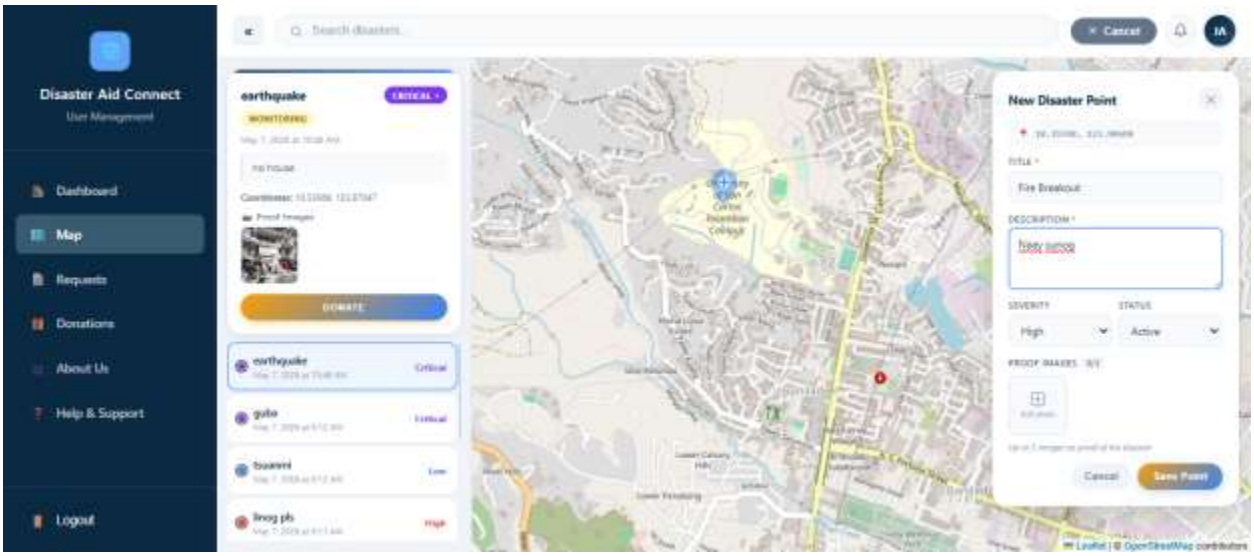
2. Register page — successful account creation



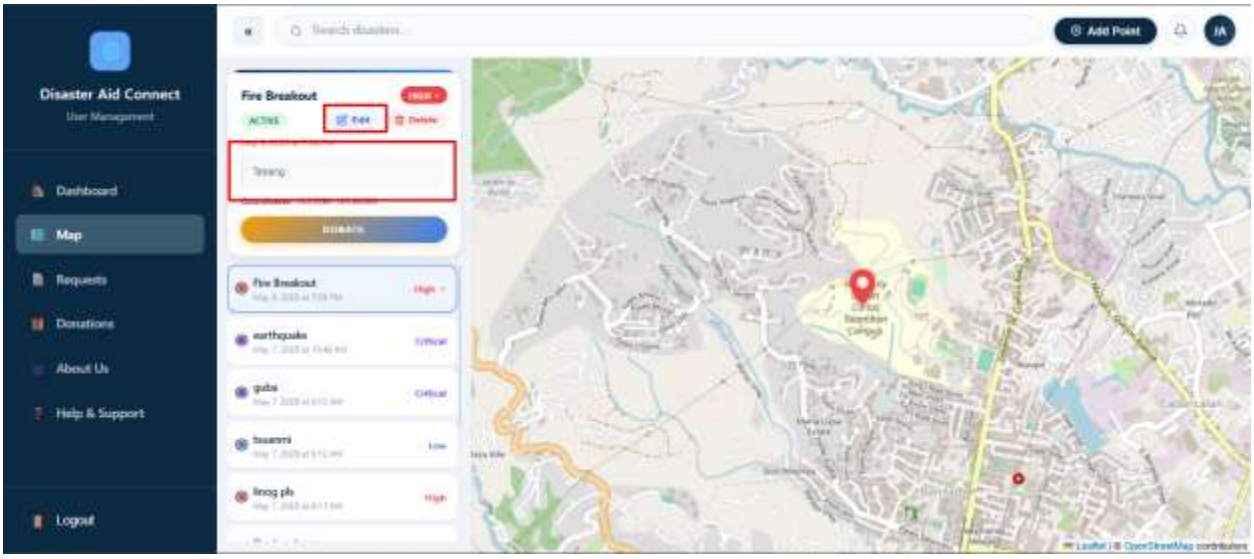
3. Map page — disaster markers loading on OpenStreetMap



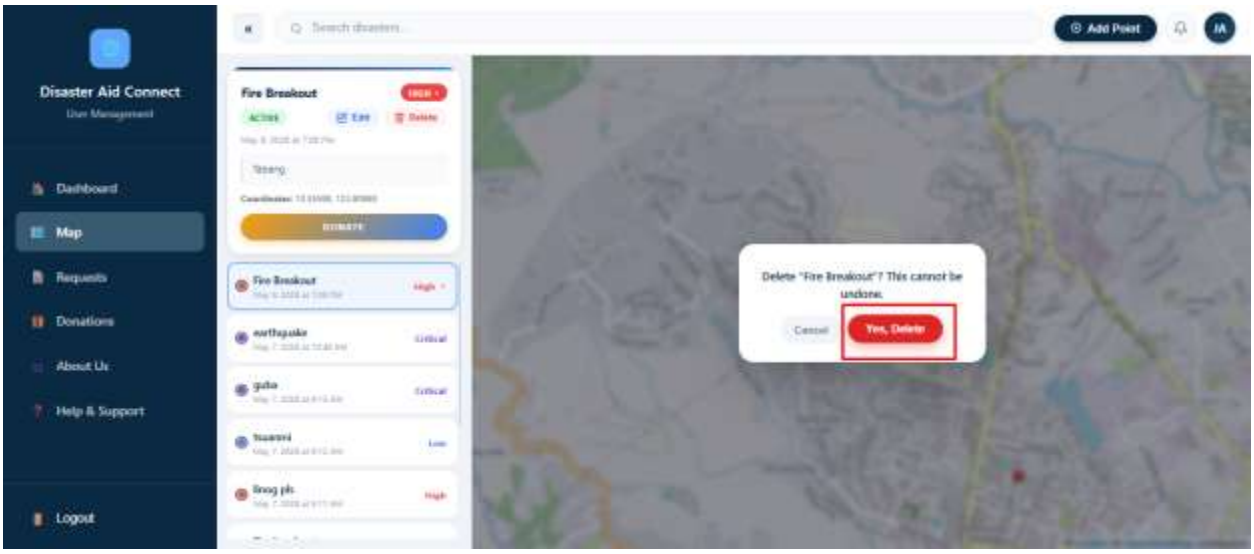
4. Add Point form — form filled and saved, new marker visible



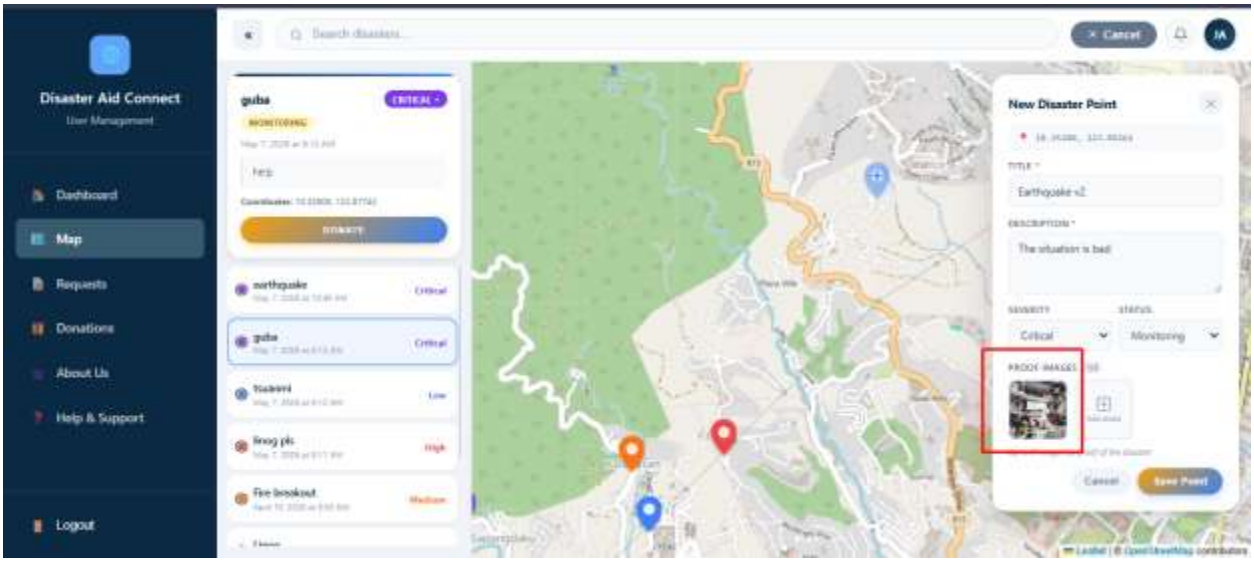
5. Edit Point form — updated details reflected in detail card



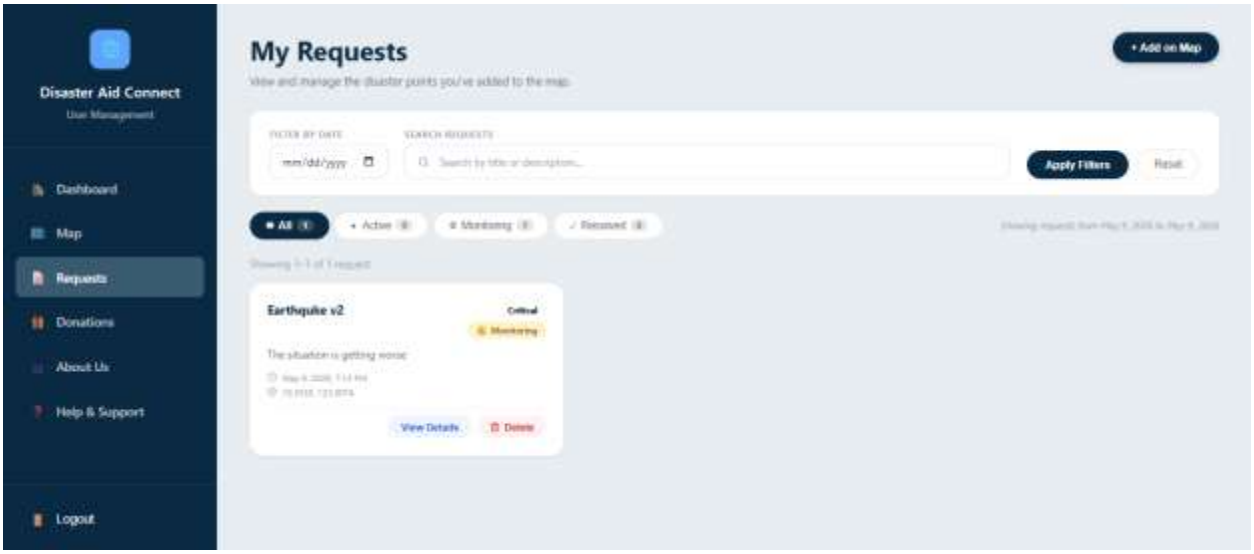
6. Delete confirmation dialog — point removed after confirm



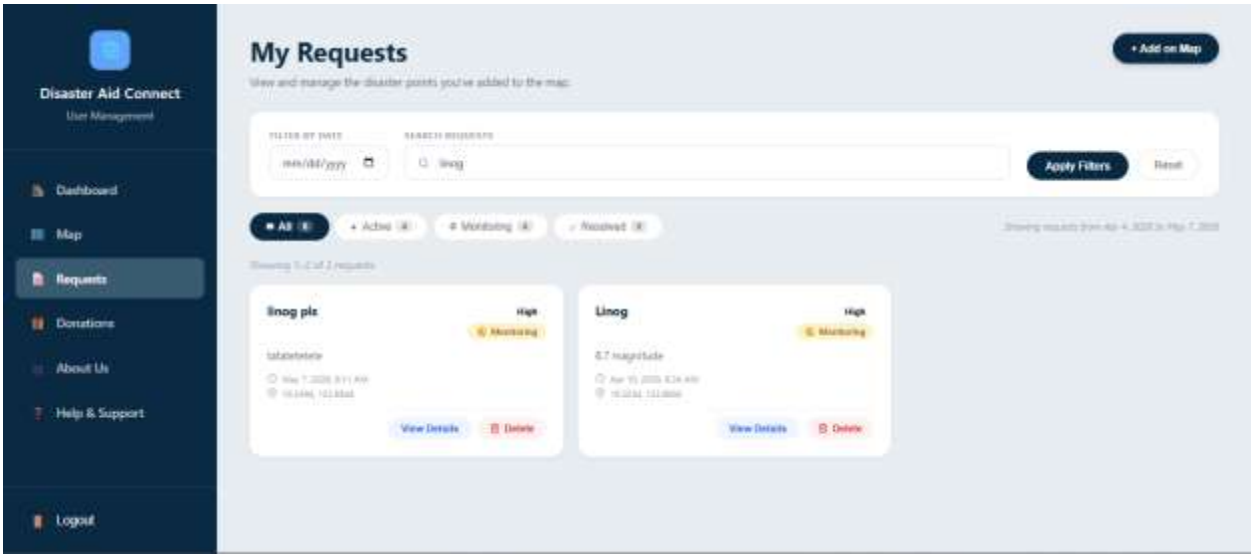
7. Proof image upload — thumbnails visible in detail card



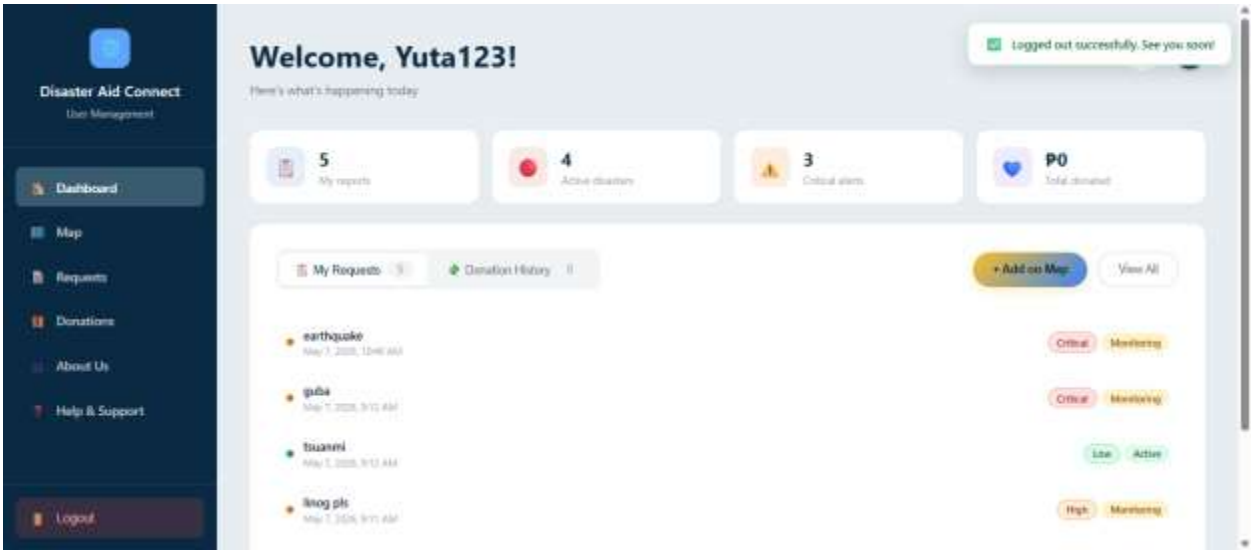
8. Requests page — user-specific disaster points listed



9. Search and filter — results filtered by keyword and status



10. Logout — session cleared, redirected to login



Test Logs

Browser console logs were monitored throughout all test cases.No JavaScript errors or unhandled promise rejections were observed during testing.

Network requests were inspected via Chrome DevTools (Network tab).All Supabase API calls returned HTTP 200 status codes.Authentication tokens were correctly issued on login and cleared on logout, confirmed via Application > Local Storage in DevTools.

Coverage Report

Manual test coverage achieved across all implemented features:

Authentication	: 100% (register, login, OAuth, logout)
Disaster Map	: 100% (view, add, edit, delete, images)
Requests Page	: 100% (list, search, filter, delete)
Navigation/Routing	: 100% (protected routes, sidebar links)
About & Help Pages	: 100% (content loads correctly)

Automated Test Results

Formal automated testing frameworks (Jest, Cypress) were not configured in this iteration. All 15 test cases were executed manually and passed successfully with zero failures or regressions detected post-refactoring.

Future iterations will include Jest unit tests for service functions and Cypress end-to-end tests for critical user flows.

Regression Test Results

Test Environment:

Browser: Google Chrome (latest)

OS: Windows 11

Frontend: http://localhost:3000

Backend: http://localhost:8080

Database: Supabase (PostgreSQL)

Date: May 9, 2026

Summary:

Total Test Cases : 15

Passed : 15

Failed : 0

Blocked : 0

Pass Rate : 100%

Result per Test Case:

TC-01 User Registration **PASS**

TC-02 Login - Valid Credentials **PASS**

TC-03 Login - Invalid Credentials **PASS**

TC-04 Google OAuth2 Login **PASS**

TC-05 View Disaster Map **PASS**

TC-06 Add Disaster Point **PASS**

TC-07 Edit Disaster Point (Owner) **PASS**

TC-08 Delete Disaster Point (Owner) **PASS**

TC-09 Non-Owner Restriction **PASS**

TC-10 Upload Proof Images **PASS**

TC-11 View My Requests **PASS**

TC-12 Search and Filter Requests **PASS**

TC-13 Delete from Requests Page **PASS**

TC-14 Logout **PASS**

TC-15 Protected Route Redirect **PASS**

Conclusion:

All functional requirements verified successfully after vertical Slice Architecture refactoring. No regressions detected.

Issues Found

Issues Found During Regression Testing

Issue-01

Title: Import path errors after frontend refactoring

Severity: High (blocked compilation)

Description:

After moving About.js and Help.js into src/shared/pages/, the import paths for Dashboard.css and Sidebar.js were incorrect, causing webpack to fail.

Root Cause:

Relative import paths were not updated to reflect the new deeper folder depth.

Issue-02

Title: ProtectedRoute supabaseClient path incorrect

Severity: High (blocked compilation)

Description:

ProtectedRoute.js used ../../src/supabaseClient which

resolved outside the src directory.

Root Cause:

File moved to src/shared/components/ without updating

the import path.

Issue-03

Title: useAuth hook import missing in Map.js

Severity: High (blocked compilation)

Description:

Map.js imported useAuth from ../../hooks/useAuth but

the hooks folder did not exist at that path.

Root Cause:

Hook path not aligned with new folder structure.

Fixes Applied

Fix-01 (resolves Issue-01) File: src/shared/pages/About/About.js
src/shared/pages/Help/Help.js

Change: Updated Dashboard.css import
from ../../features/Dashboard/Dashboard.css
to ../../../features/dashboard/Dashboard.css

Fix-02 (resolves Issue-02) File: src/shared/components/ProtectedRoute.js
Change: Updated supabaseClient import
from ../../src/supabaseClient
to ../../supabaseClient

Fix-03 (resolves Issue-03) File: src/features/disaster/Map.js
Change: Updated useAuth import to correct relative path matching the actual hooks folder location inside src/hooks/useAuth.js

All three fixes were committed and pushed to the refactor branch. After applying fixes, npm start compiled successfully with zero errors.